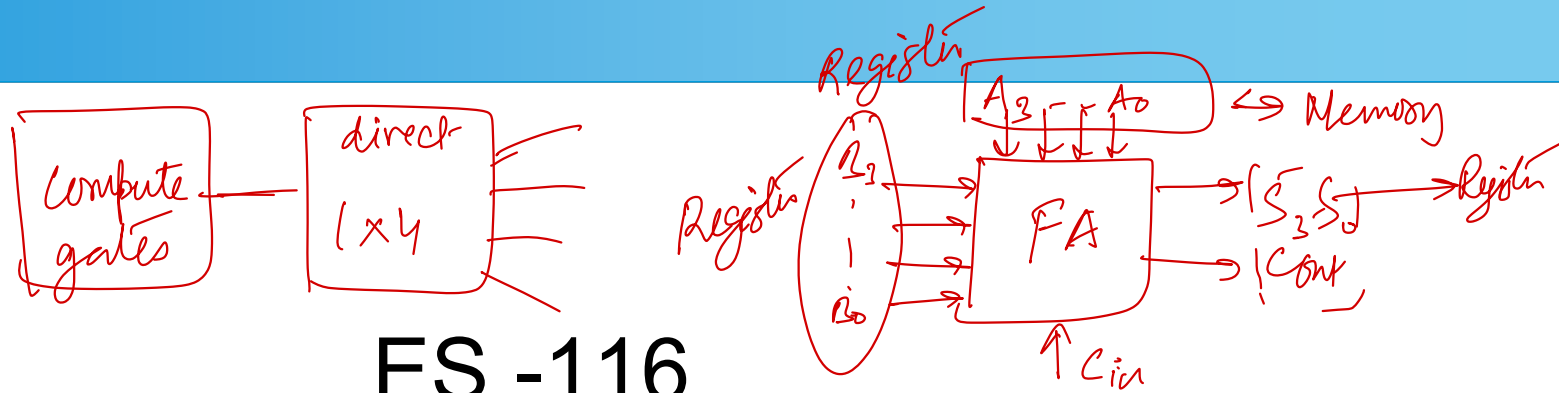




ES -116

Principles and Applications of Electrical Engineering

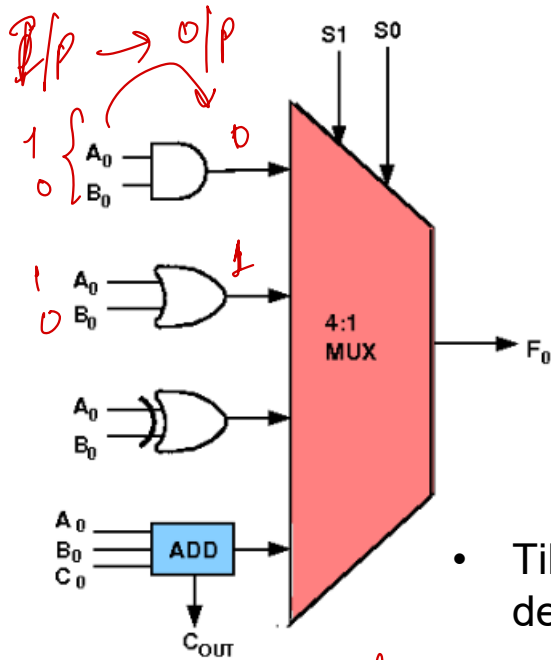
Semester II, 2025-26

How do Processors Store Data?

Reference book: Digital Systems Book by Tocci

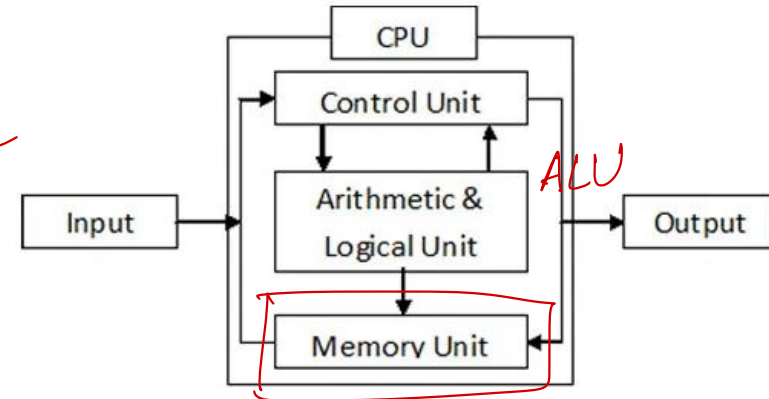
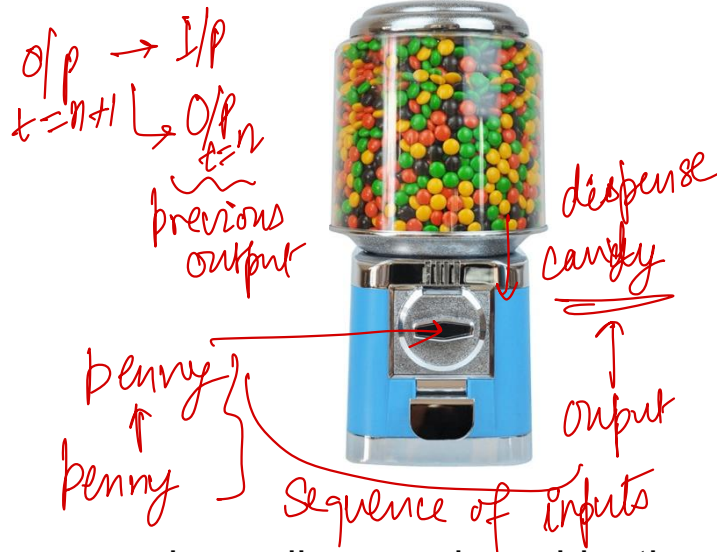
Digital computers: Beyond ALU

Example: 1-bit ALU unit



Combinational
Circuit

Relatable example: Penny Slot Machine

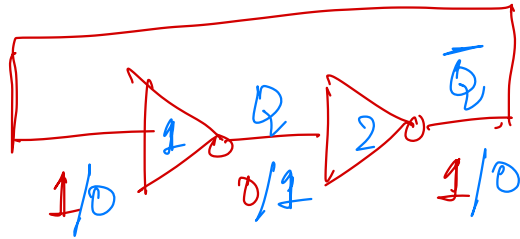
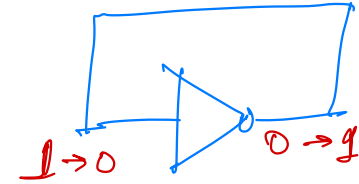
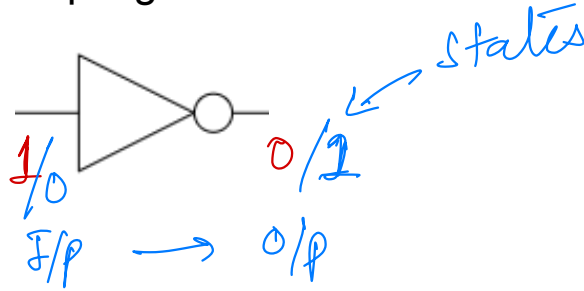


Block diagram of a *digital computer*

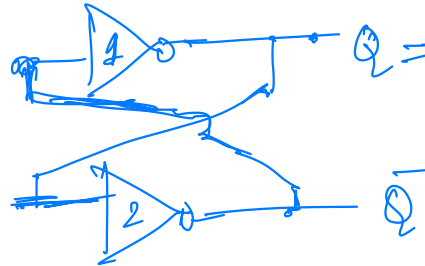
- Till now, we have discussed combinational logic circuits, where its output only depends on the applied inputs i.e. it doesn't have any memory.
- Next, we will discuss a specific type of digital circuits called "sequential logic circuits" that does have a memory i.e. its output depends on both applied inputs and its past output.

Simplest digital 1-bit memory element

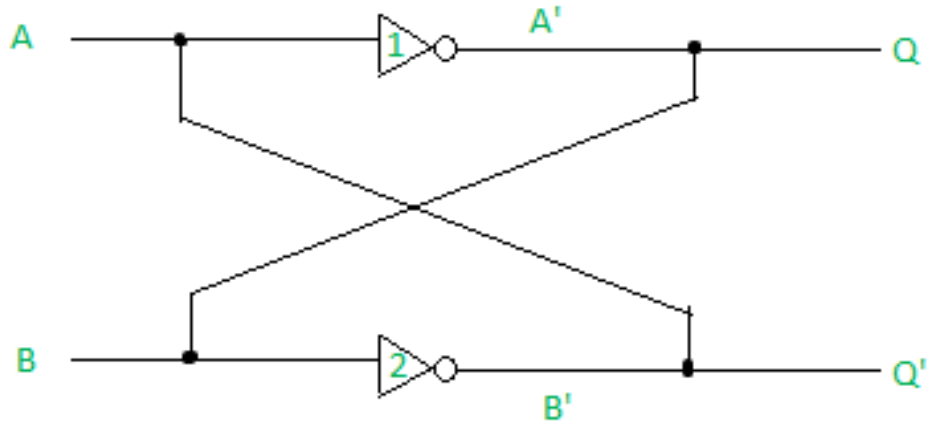
How to trap signal in time?



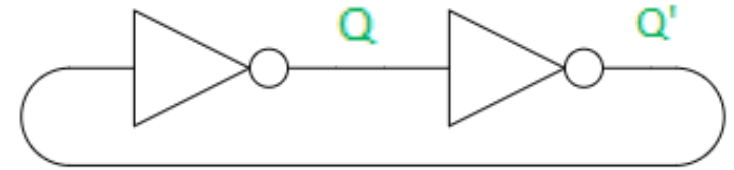
trap signal in time



Simplest digital 1-bit memory element



≡



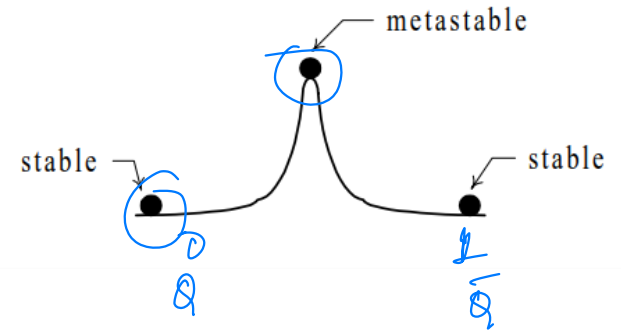
bistate memory element

SET condition:

RESET condition:

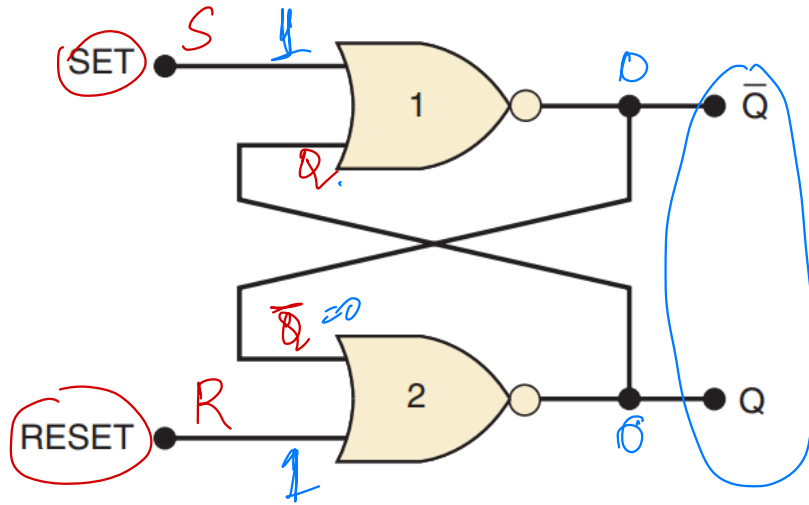
we can not directly SET 0/1 at Q!

Analogy



Capability to SET and RESET the memory: NOR Latch

NOR Latch with SET and RESET inputs



SR latch

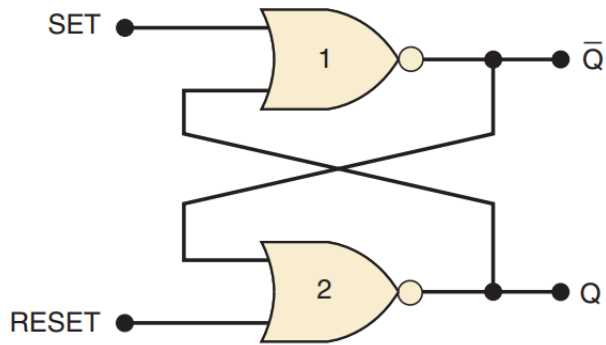
Memory element
What is the
initial condition
of Q?

Truth Table

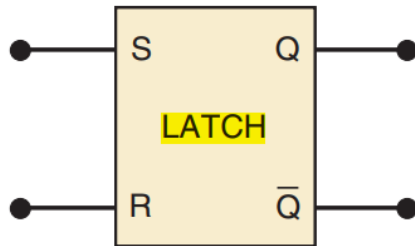
S	R	Q_n	Q_{n+1}	$\bar{Q}_{n+1}$
0	0	0/1	Q_n	$\bar{Q}_n$
0	1	0/1	0	1
1	0	Q_n	1	0
1	1	Q_n	Ambiguous state	

NOR Latch

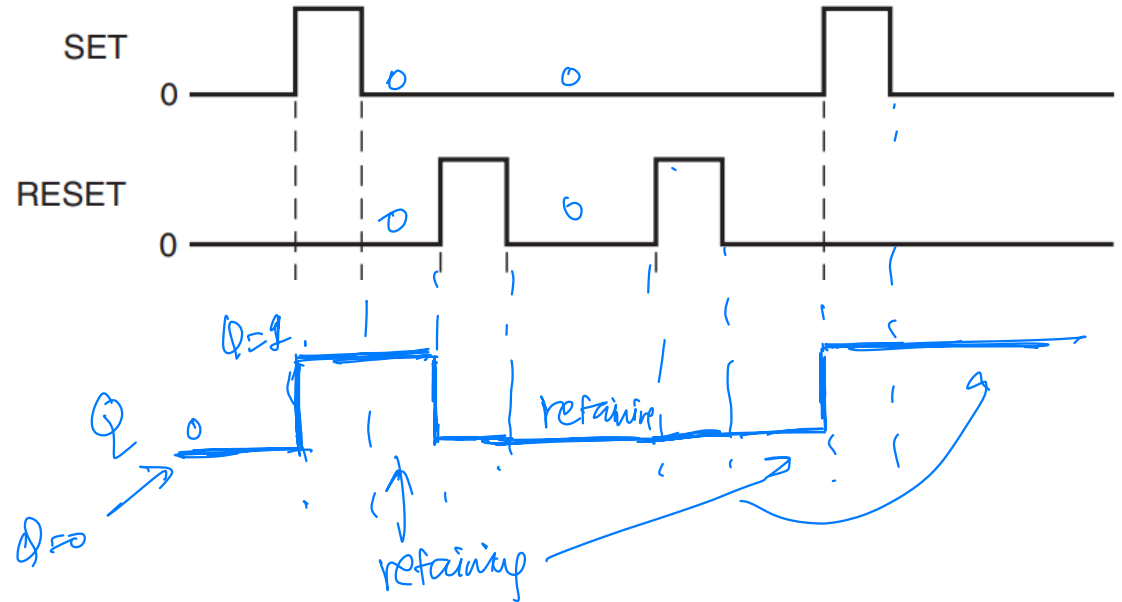
NOR Latch



NOR Latch Symbol (Active HIGH inputs)



Assuming initial Q as 0, draw the Q waveform.



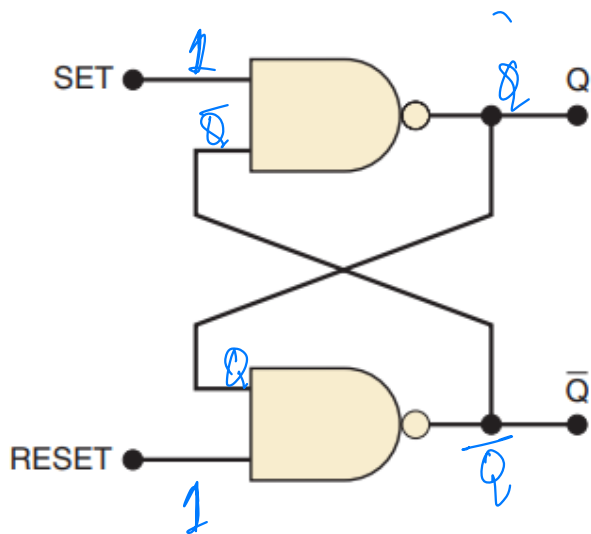
Capability to SET and RESET the memory: NAND Latch

Remember the duality property of Boolean logic?

$NOR \leftrightarrow NAND$

Truth Table

NAND Latch with SET and RESET inputs

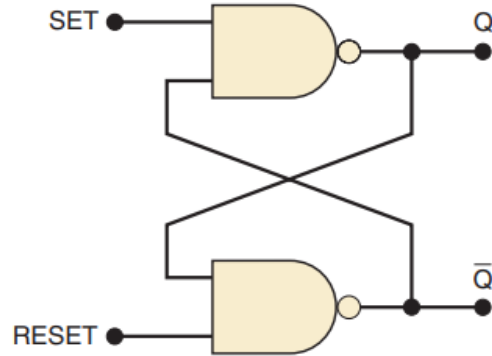


Race condition
(Ambiguous)

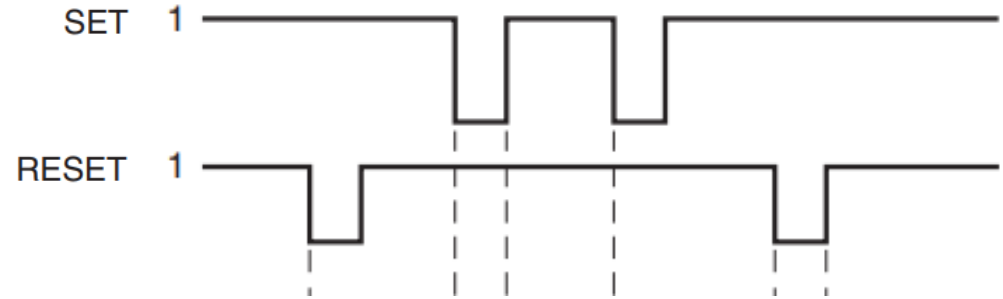
S	R	Q_n	Q_{n+1}	$\overline{Q_{n+1}}$
0	0		1	1
0	1	Q_n	1	0
1	0	Q_n	0	1
1	1	Q_n	Q_n	$\overline{Q_n}$

NAND Latch

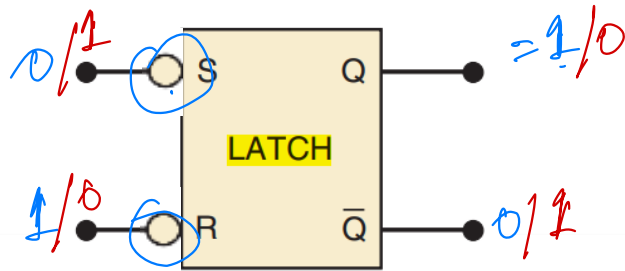
NAND Latch



Assuming initial Q as 0, draw the Q waveform.

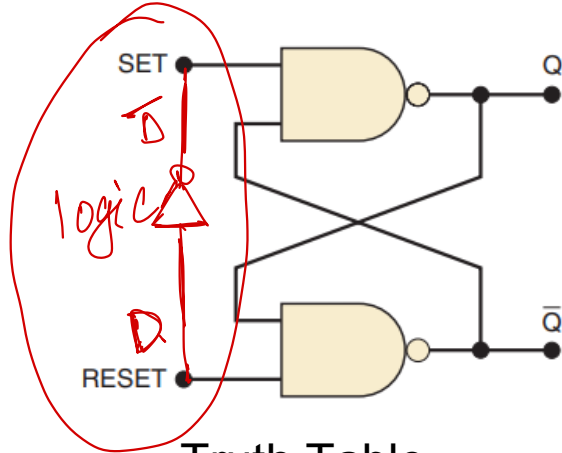


NAND Latch Symbol (Active LOW inputs)



Removing undefined state in SR Latch

How can you get rid of the “invalid” input conditions?

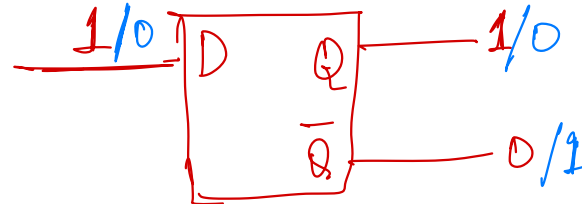


Truth Table

Set	Reset	Output
1	1	No change
0	1	$Q_n = 1$
1	0	$Q_n = 0$
0	0	Invalid *

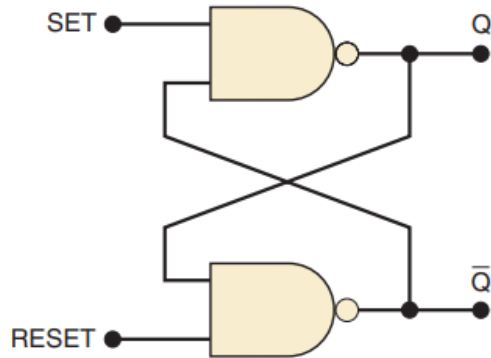


$\bar{D}(R)$	$\bar{D}(S)$	
1	0	$Q_{n+1} = 1$
0	1	$Q_{n+1} = 0$



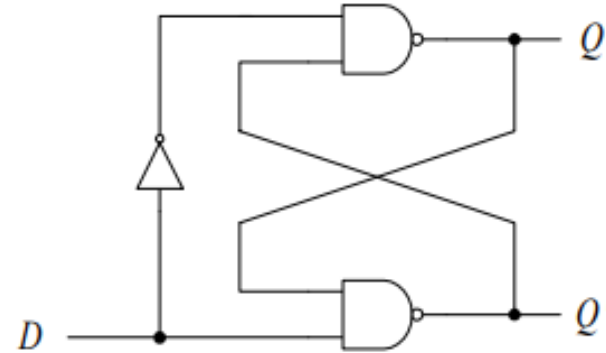
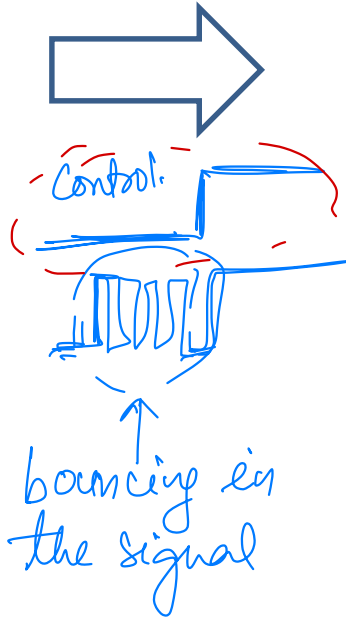
D latch

Removing undefined state in RS latch: The D-Latch



Truth Table

Set	Reset	Output
1	1	No change
0	1	$Q = 1$
1	0	$Q = 0$
0	0	Invalid *

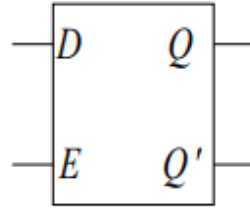
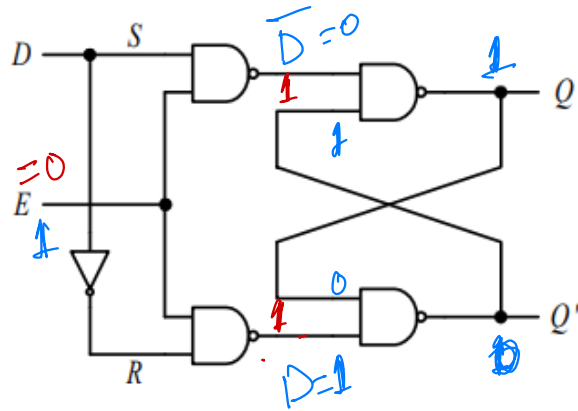


Truth Table

D	Q	$\bar{Q}$
1	1	0
0	0	1

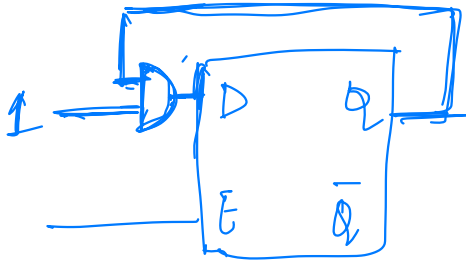
D-Latch with enable signal

With Enable signal



$E=0$ [retaining]

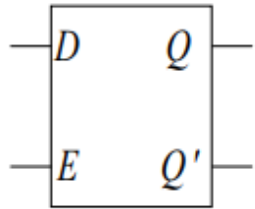
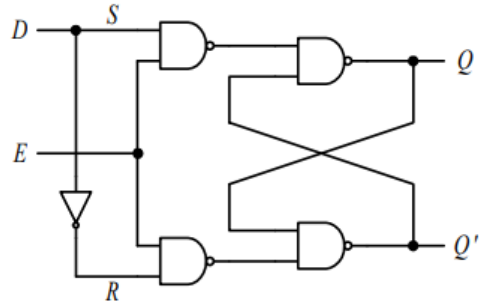
$E=1$ $\left\{ \begin{array}{l} D=0 \rightarrow Q_{n+1} \rightarrow 0 \\ D=1 \rightarrow Q_{n+1} \rightarrow 1 \end{array} \right.$



Truth Table

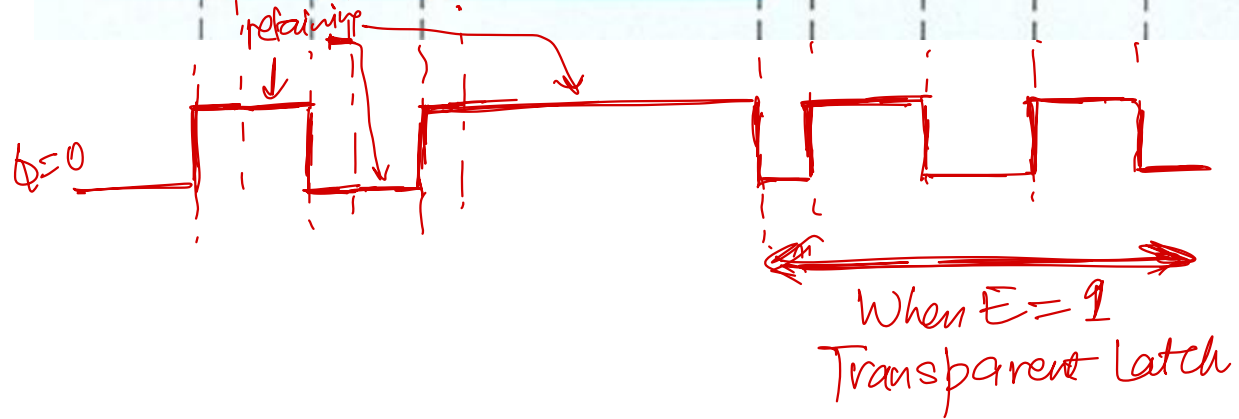
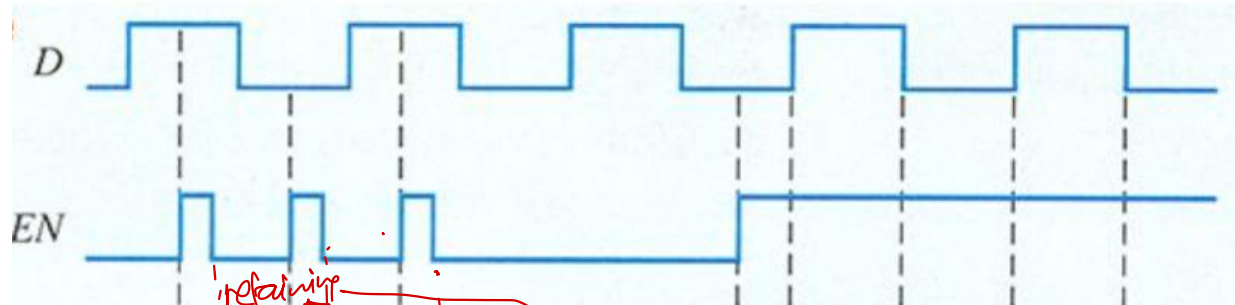
E	D	Q_n	Q_{n+1}	$\overline{Q_{n+1}}$
0	0		Q_n	$\overline{Q_n}$
0	1		Q_n	$\overline{Q_n}$
1	0	Q_n	0	1
1	1	Q_n	1	0

D-Latch: Transparent latch

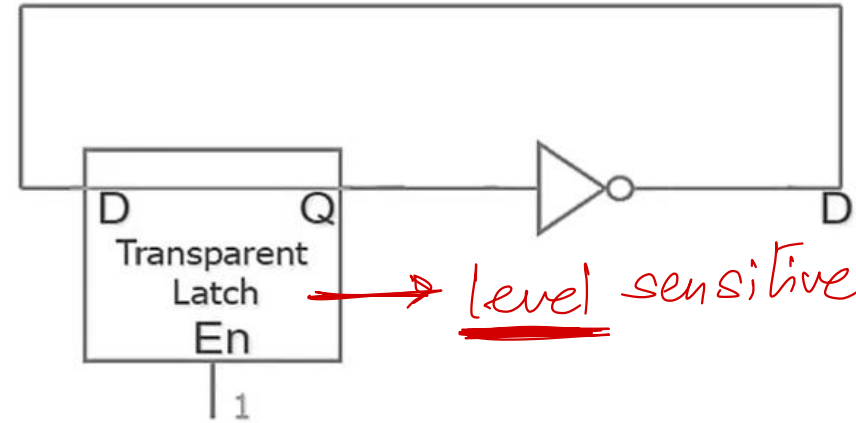
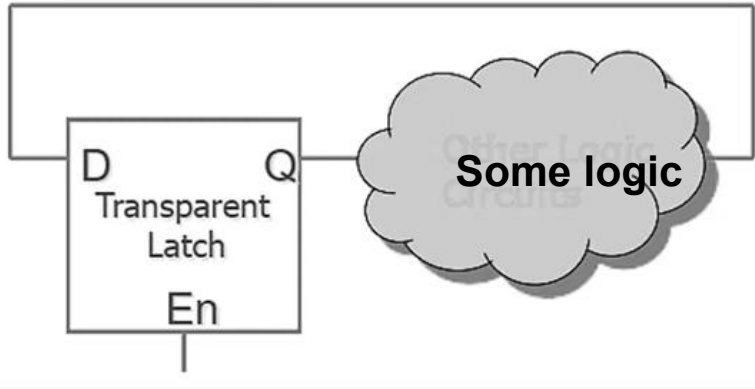


Inputs		Output
EN	D	Q
0	X	Q_0 (no change)
1	0	0
1	1	1

Draw the waveform for Q?

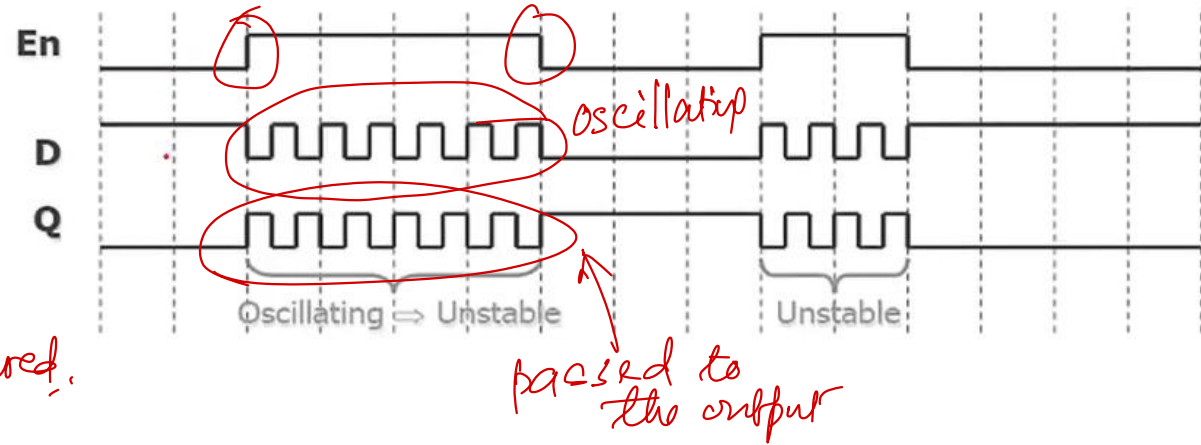


Problem in D-latch



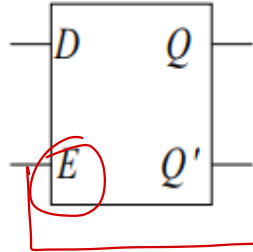
level sensitive

requirement
input is read only when
En goes from $0 \rightarrow 1$ or $1 \rightarrow 0$
(edge of the enable) $\rightarrow$ edge
sensitive or edge triggered.



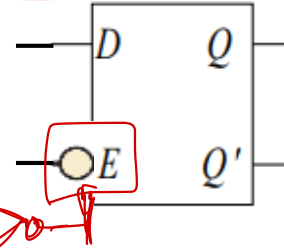
D-Latch: active HIGH and active LOW enable signal

Active HIGH enable signal



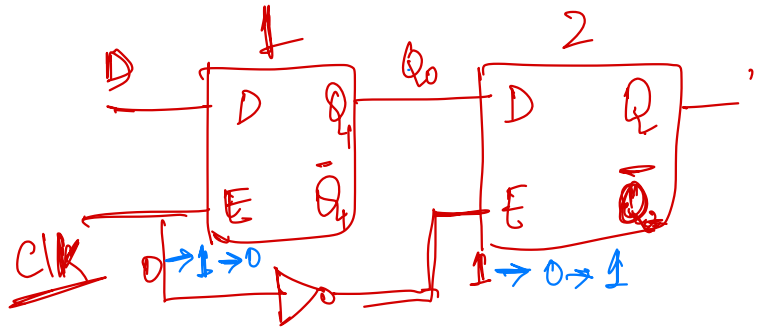
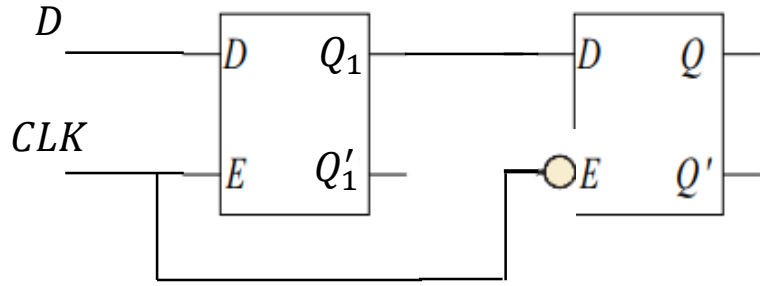
Inputs		Output
EN	D	Q
0	X	Q_0 (no change)
1	0	0
1	1	1

Active LOW enable signal

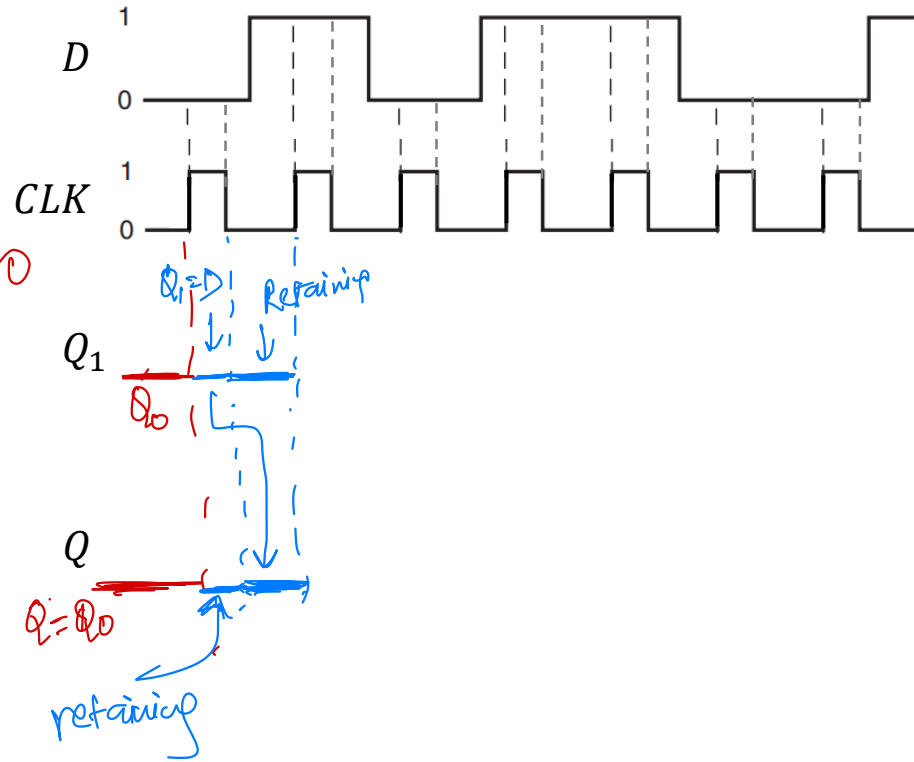


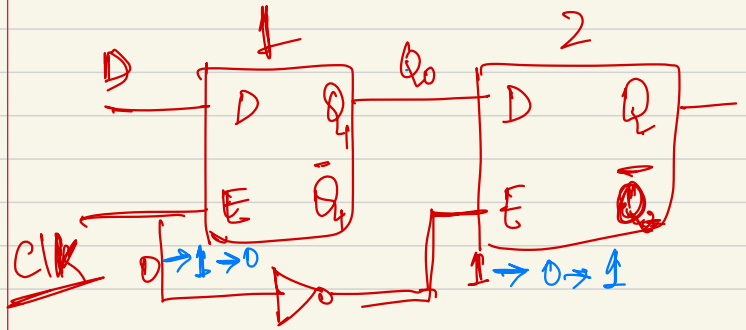
Inputs		Output
EN	D	Q
0	0	1
1	X	Q_0 (no change)

Lets combine both

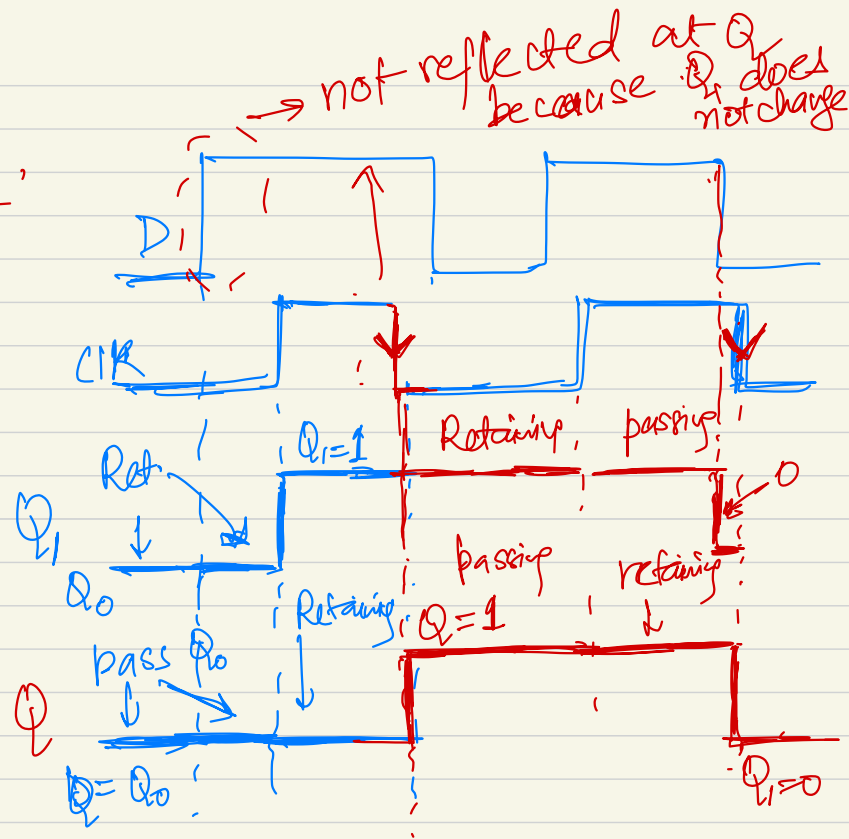
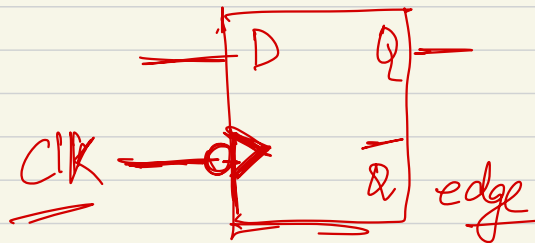


$$Q_0 = 0$$

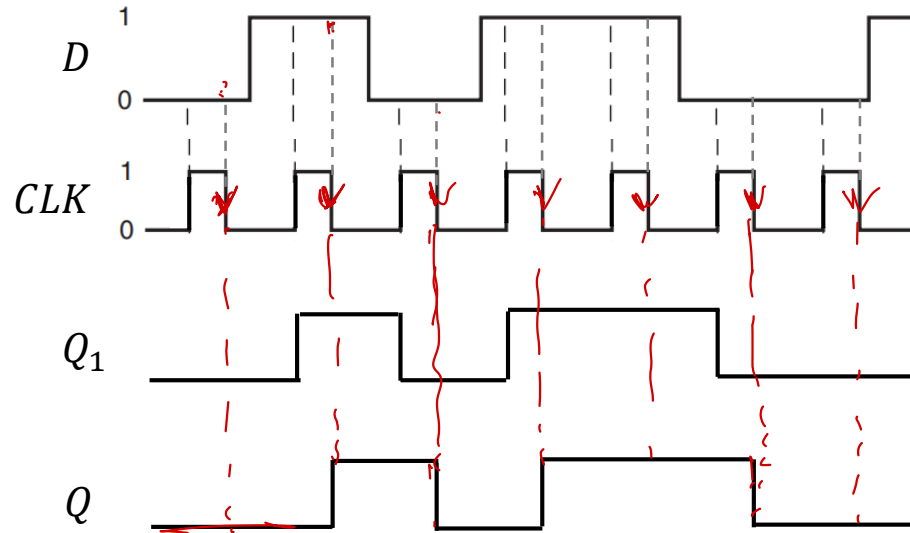
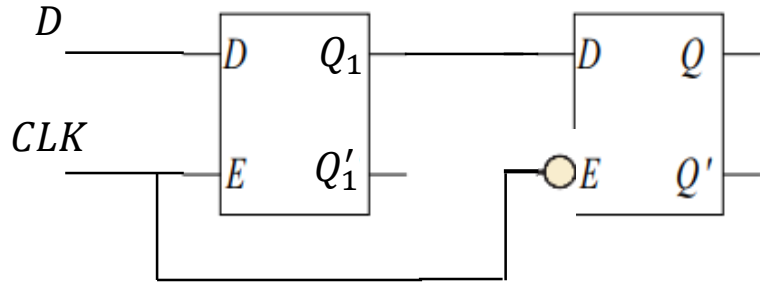




edge triggered device

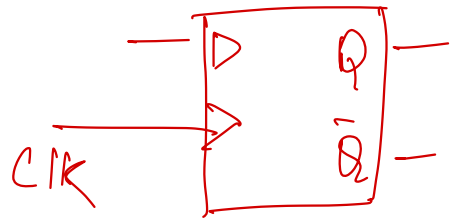
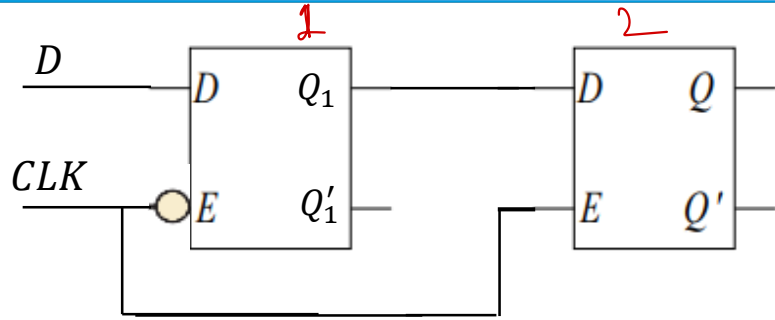


~~Master-Slave~~ (Sensitive name) D Latches

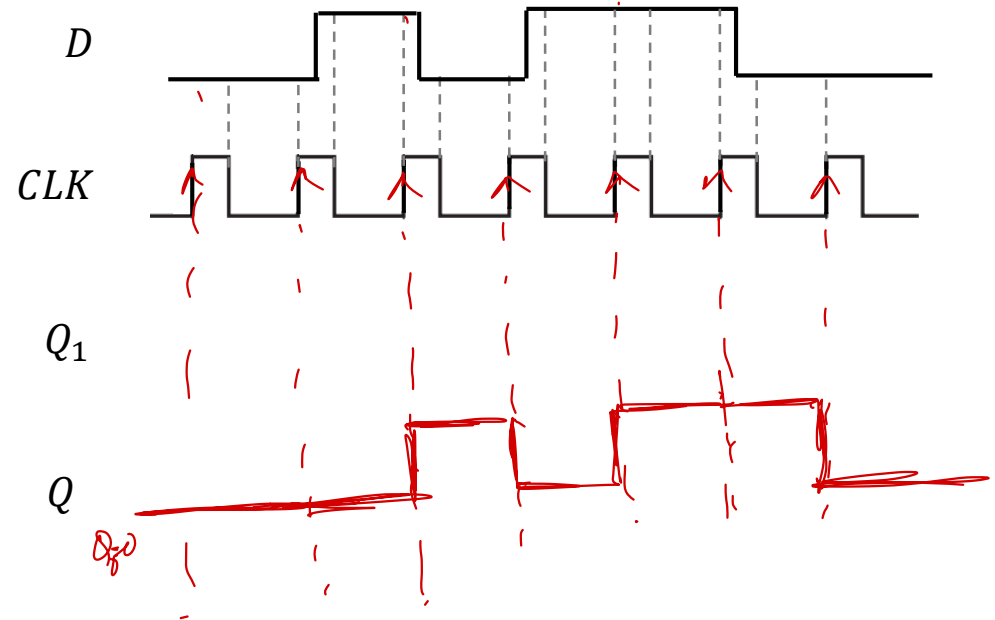


Flip Flop (FF) passes the input just before the CLK transition from 1 $\rightarrow$ 0 or at negative edge of the clock.

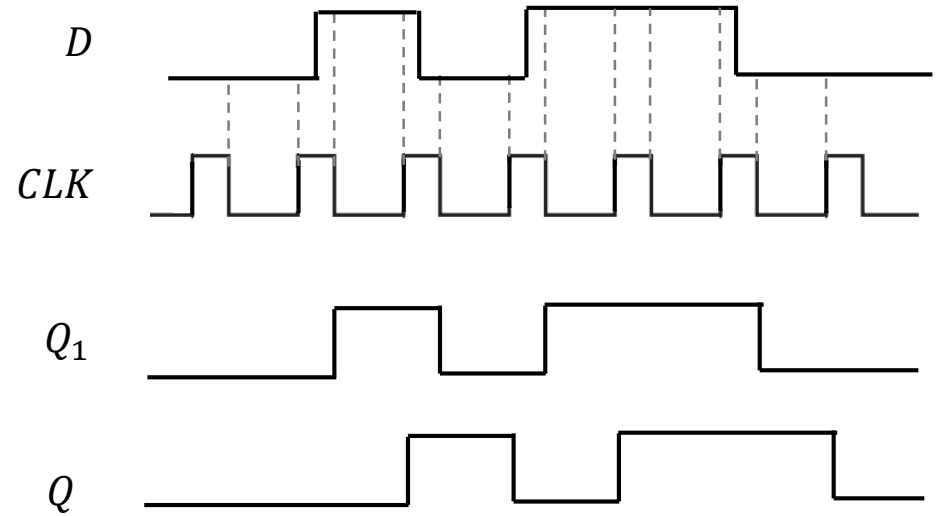
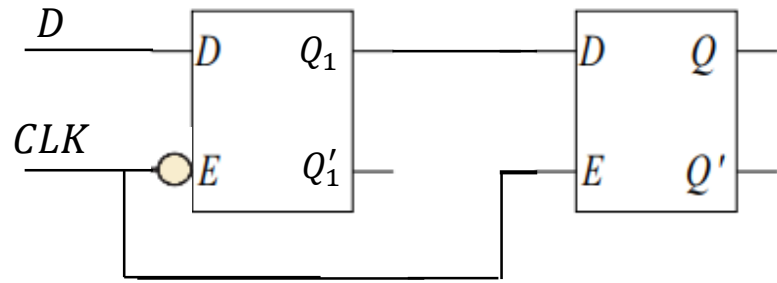
Opposite combination



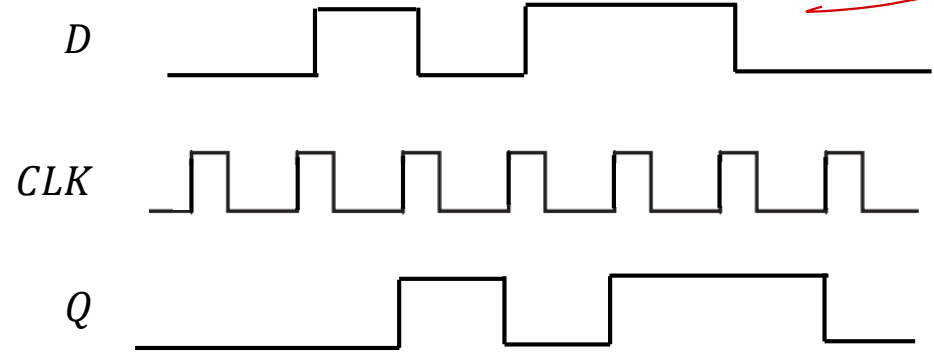
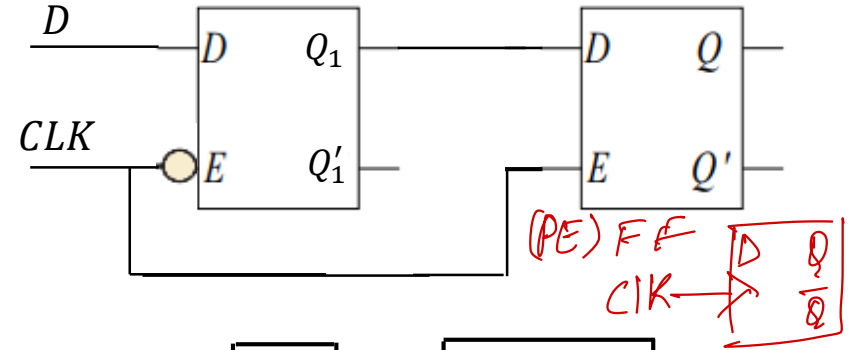
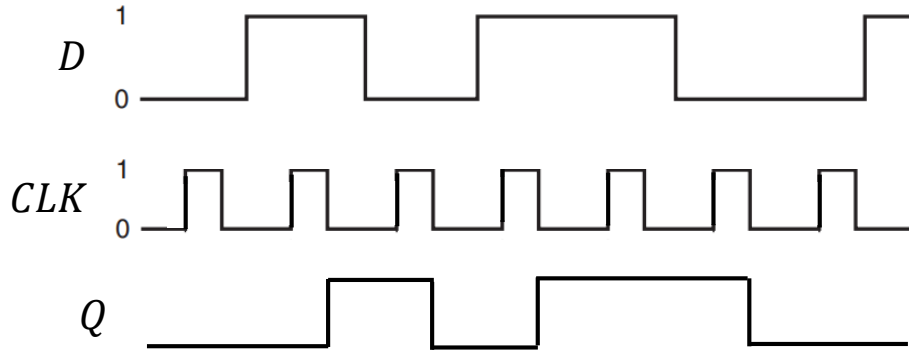
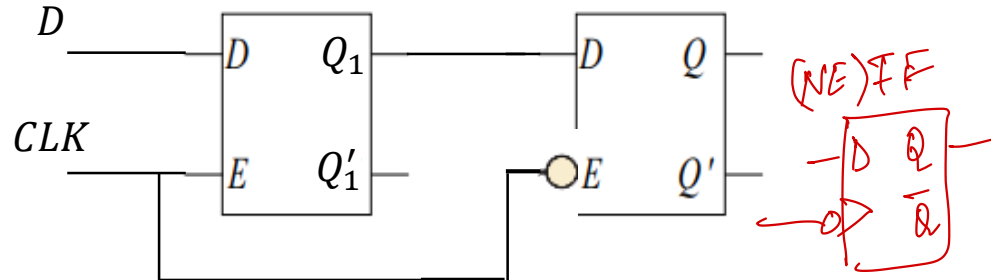
CLK (positive edge)
triggered FF.



Master-Slave D Latches

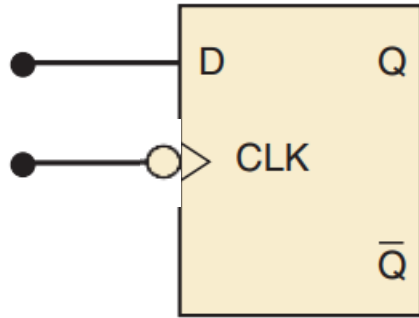


Observations

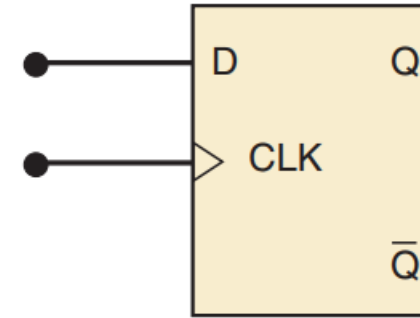


Using the blocks as edge-triggered element

Falling (Negative) Edge Triggered

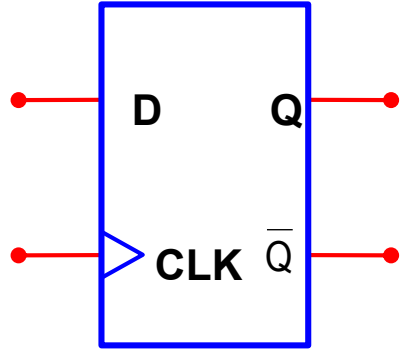


Rising (Positive) Edge Triggered



D Flip-Flop (D FF)

Positive Edge Trigger

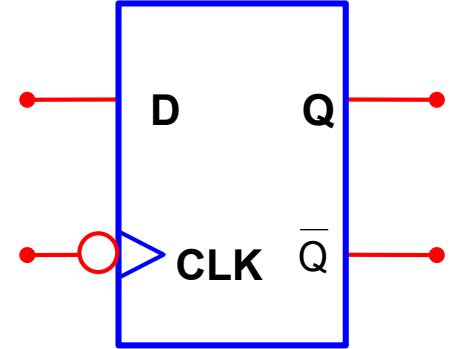


D	CLK	Q	$\bar{Q}$
0	↑	0	1
1	↑	1	0

↑ : Rising Edge of Clock

*latch → level sensitive
FF → edge sensitive*
memory element

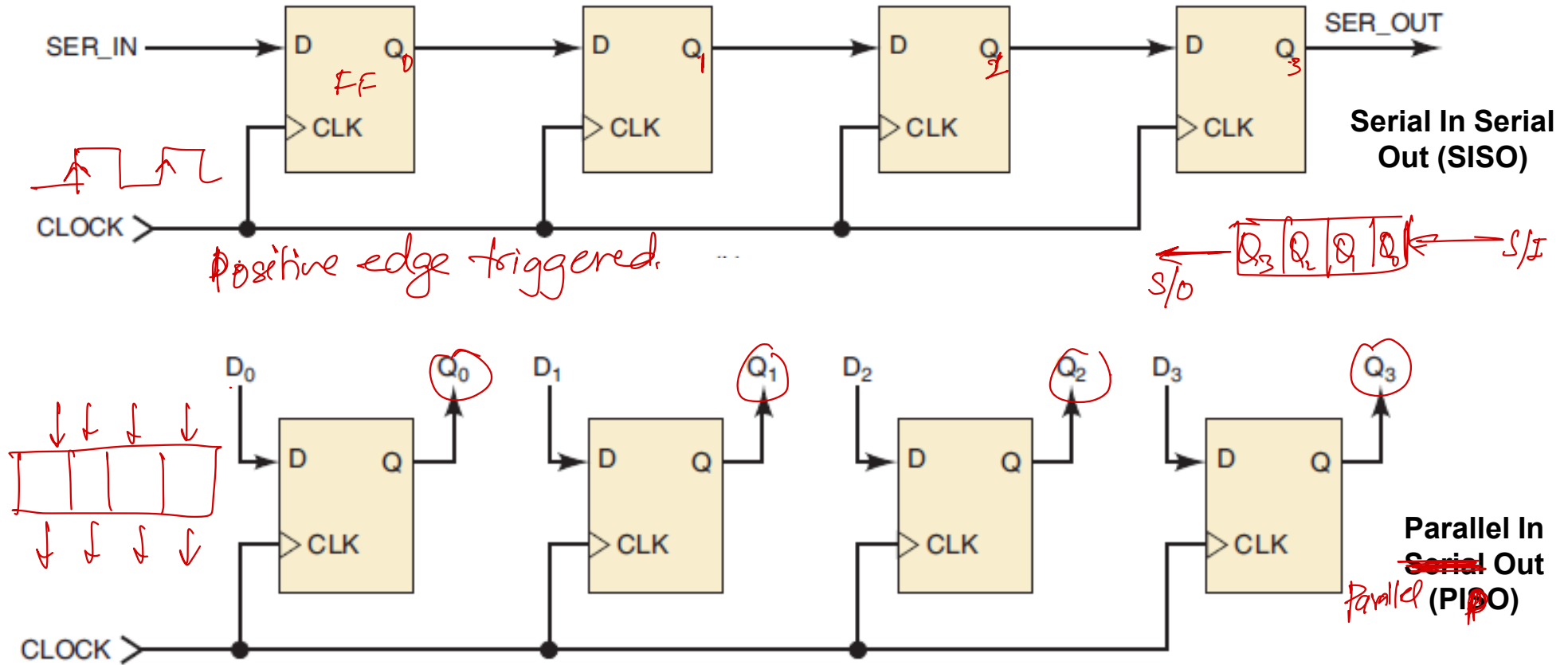
Negative Edge Trigger



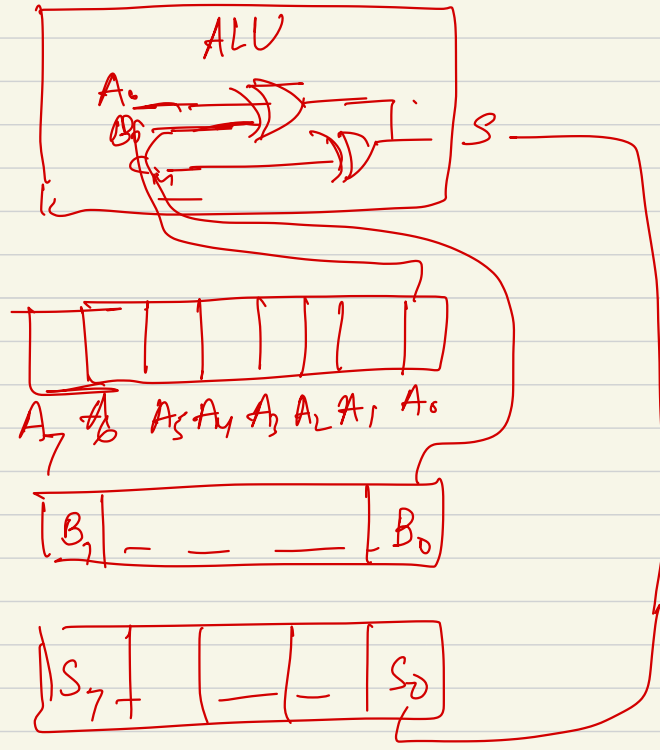
D	CLK	Q	$\bar{Q}$
0	↓	0	1
1	↓	1	0

↓ : Falling Edge of Clock

Application in Multi-bit Memory (Registers)

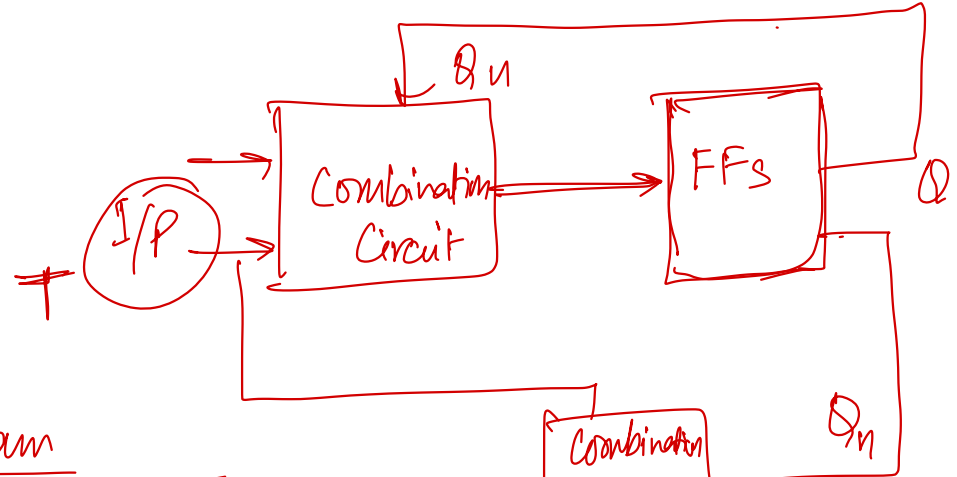
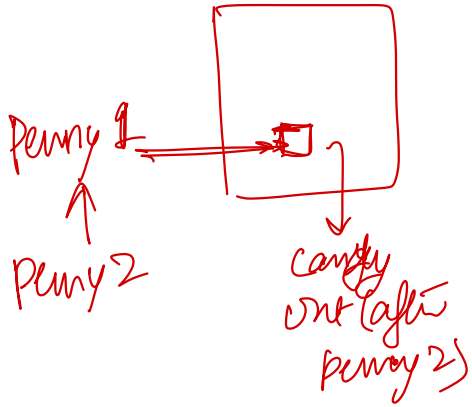


Registers



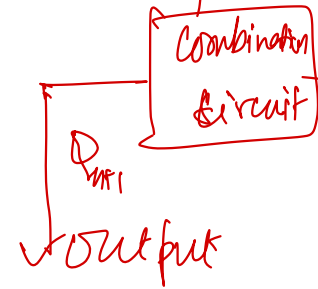
Introduction to Finite State Machines

Penny Slot Machine

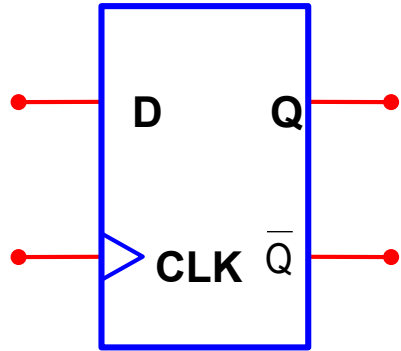


State diagram

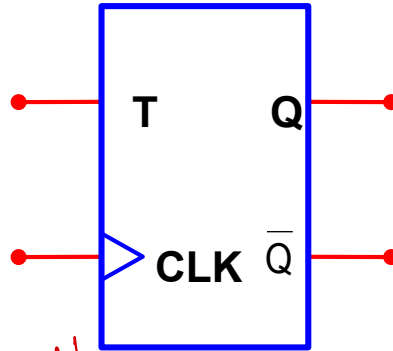
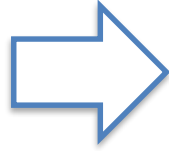
Q_n	Q_{n+1}	What input do I need to give?
0	1	
1	0	



Making D Flip-Flop Interesting?



↑ clk edge



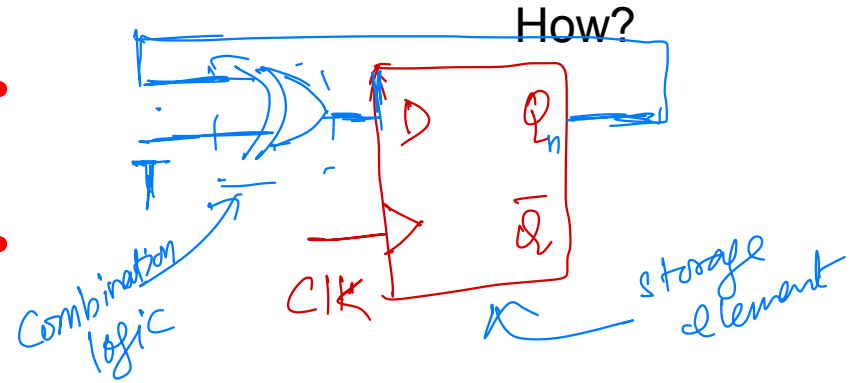
State Table

D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

clk=0, $Q_{n+1} = Q_n$

T	Q_n	Q_{n+1}	D
0	0	0	0
0	1	1	1
1	0	1	1
1	1	0	0

Toggle Flip Flop



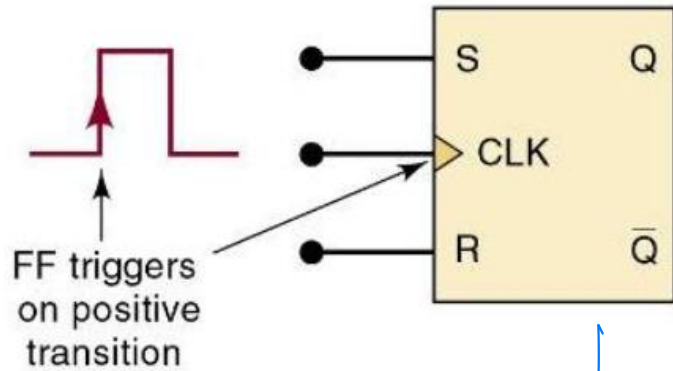
When $T=0$, $Q_{n+1} = Q_n$ } Toggle
 When $T=1$, $Q_{n+1} = \overline{Q_n}$ } FF.

$$D = f(T, Q_n)$$

T	Q_n	D
0	0	0
0	1	1
1	0	1
1	1	0

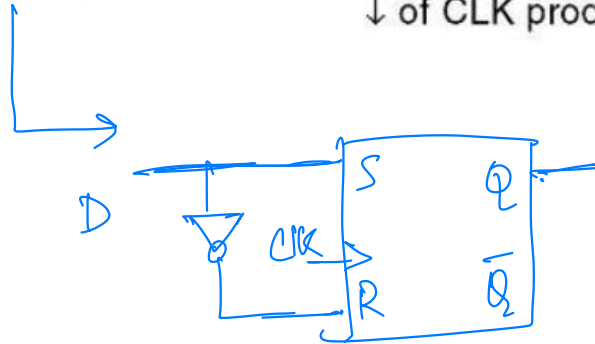
$$D = T \oplus Q_n$$

SR Flip-Flop (SR FF)



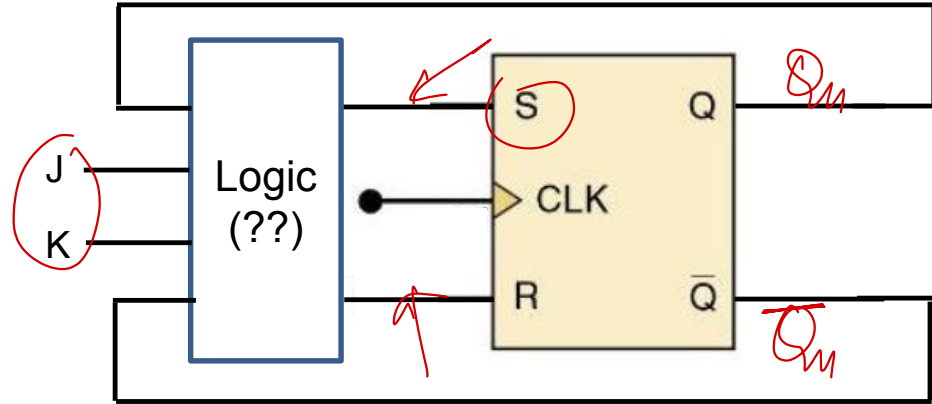
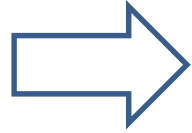
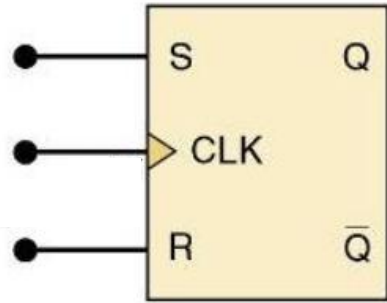
Inputs			Output
S	R	CLK	Q
0	0	↑	Q_0 (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	Ambiguous

Q_0 is output level prior to ↑ of CLK.
↓ of CLK produces no change in Q.

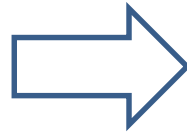


all the four states

Avoiding “ambiguous” state in S-R FF using J-K FF



Inputs			Output
S	R	CLK	Q
0	0	↑	Q_0 (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	Ambiguous

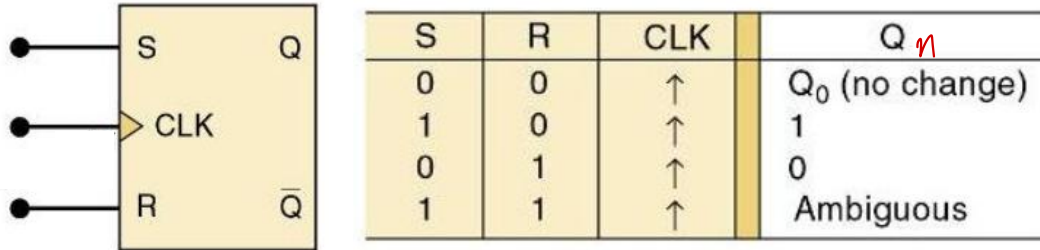


J	K	CLK	Q
0	0	↑	Q_0 (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	Q_0 (toggles)

We want output to toggle when both inputs are 1.

Methodology to build it using Excitation Tables

What we have:



What we want:

J	K	CLK	Q
0	0	↑	Q ₀ (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	$\bar{Q}_0$ (toggles)

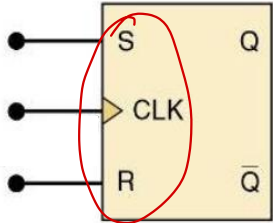
Excitation table will reveal the required inputs of the FF for all possible PRESENT to NEXT state transitions.

Excitation Table for S-R FF

Transition at FF Output	PRESENT State Q _n	NEXT State Q _{n+1}	S	R
0 → 0	0	0	0	1/0
0 → 1	0	1	1	0
1 → 0	1	0	0	1
1 → 1	1	1	1/0	0

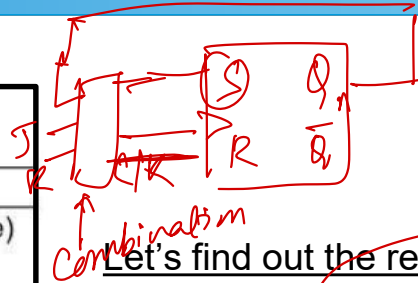
Methodology to build it using Excitation Tables

What we have:



What we want:

J	K	CLK	Q
0	0	↑	Q_0 (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	$\overline{Q_0}$ (toggles)



state Table

Let's find out the required S & R input in terms of available J, K, Q_n

Excitation Table for S-R FF

Transition at FF Output	PRESENT State Q_n	NEXT State Q_{n+1}	S	R
$0 \rightarrow 0$	0	0	0	x
$0 \rightarrow 1$	0	1	1	0
$1 \rightarrow 0$	1	0	0	1
$1 \rightarrow 1$	1	1	x	0

Q_n	J	K	Q_{n+1}	S	R
0	0	0	0	0	x
0	0	1	0	0	x
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	x	0
1	0	1	0	0	1
1	1	0	1	x	0
1	1	1	0	0	1

Logical expression simplification

Required S & R input in terms of available J, K, Q_n

Q_n	J	K	Q_{n+1}	S	R
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

Using k-map for minimizing the logic

S JK

Q_n

$\overline{Q_n}$

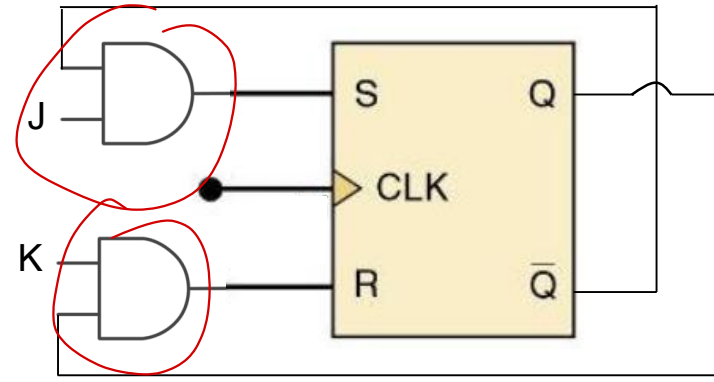
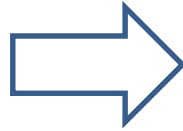
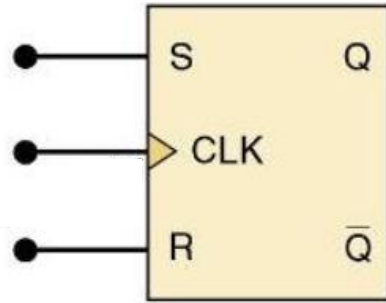
R JK

Q_n

$$S \text{ \& R} = f(J, K, Q_n)$$

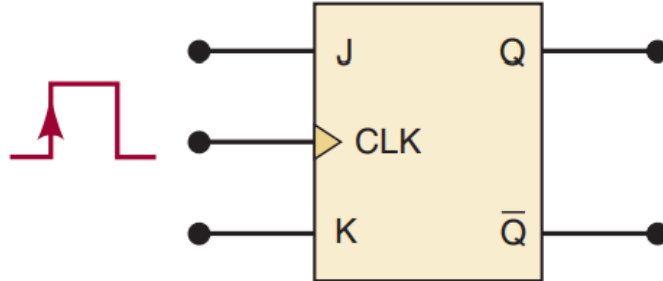
Avoiding “ambiguous” state in S-R FF using J-K FF

Named after Jack Kilby



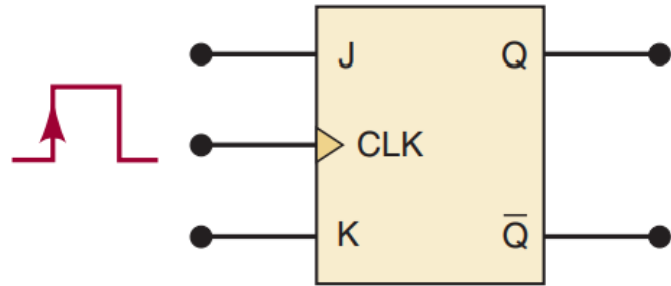
J-K FF

Symbol



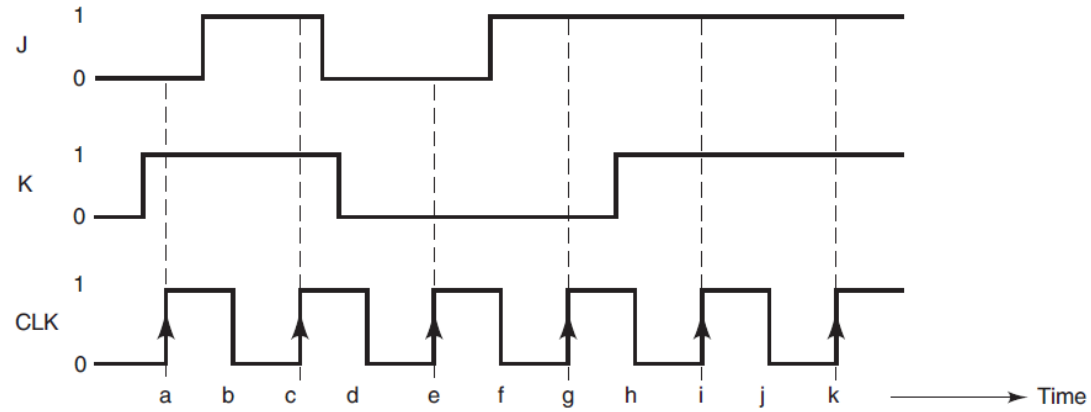
J	K	CLK	Q
0	0	$\uparrow$	Q_0 (no change)
1	0	$\uparrow$	1
0	1	$\uparrow$	0
1	1	$\uparrow$	$\bar{Q}_0$ (toggles)

J-K FF



J	K	CLK	Q
0	0	$\uparrow$	Q_0 (no change)
1	0	$\uparrow$	1
0	1	$\uparrow$	0
1	1	$\uparrow$	$\bar{Q}_0$ (toggles)

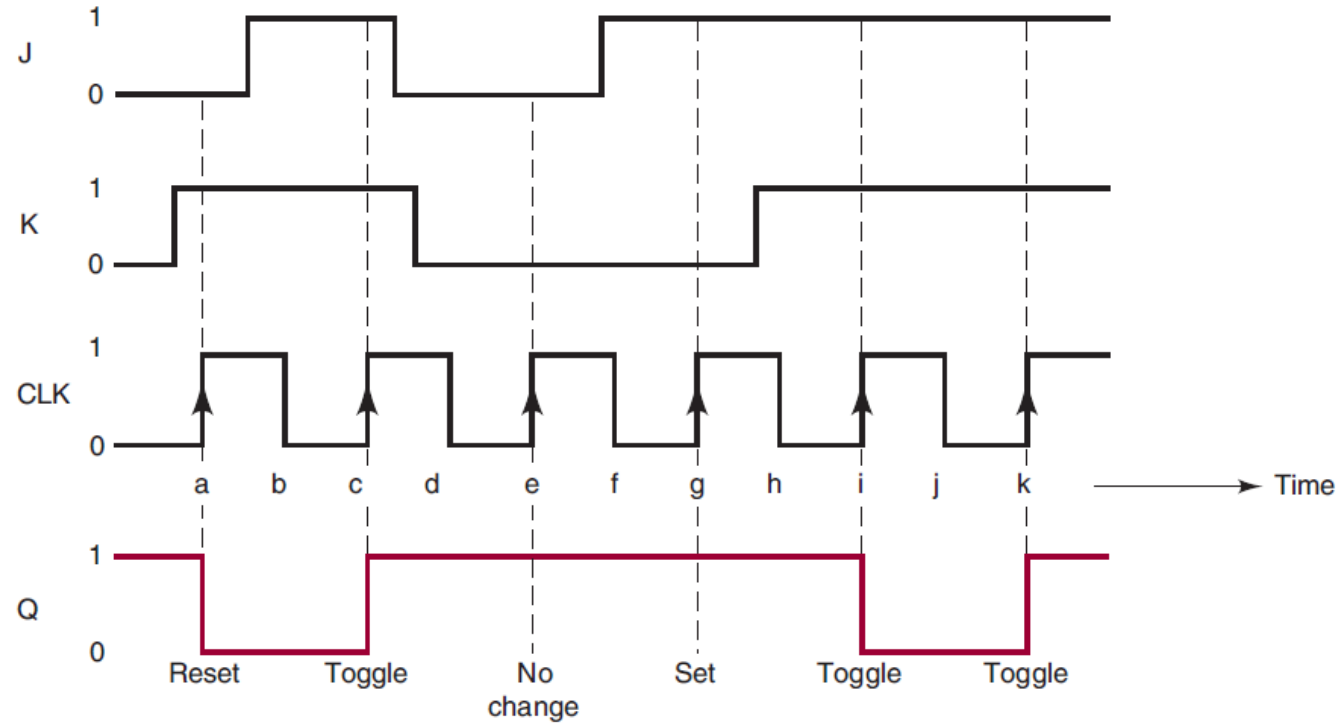
Draw the waveform for Q?



J-K FF

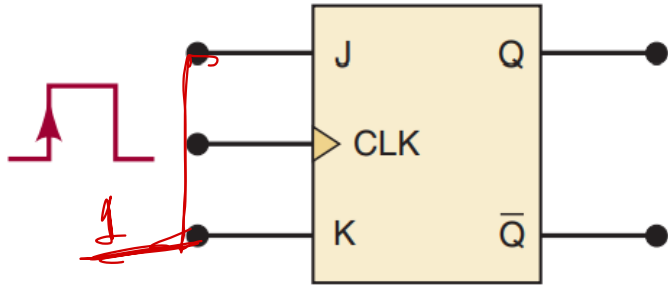
Draw the waveform for Q?

J	K	CLK	Q
0	0	↑	Q_0 (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	$\overline{Q_0}$ (toggles)



Simple application of J-K FF: Frequency division

Let's tie J & K to 1

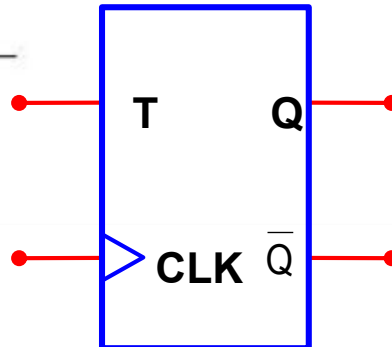


CLK



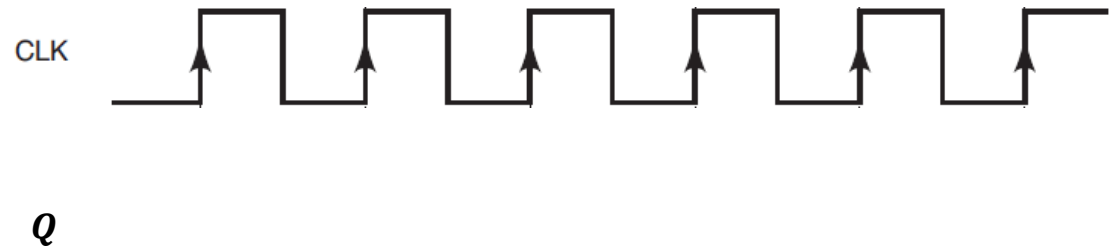
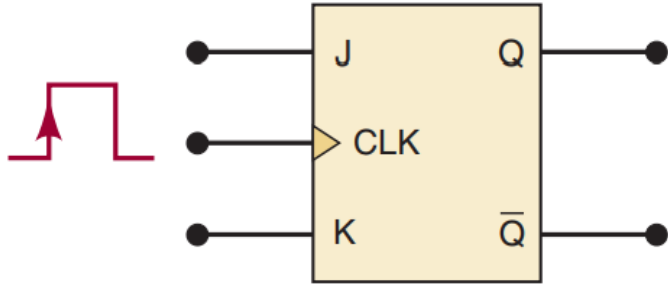
Q

J	K	CLK	Q
0	0	$\uparrow$	Q_0 (no change)
1	0	$\uparrow$	1
0	1	$\uparrow$	0
1	1	$\uparrow$	Q_0 (toggles)



Simple application of J-K FF: Frequency division

Let's tie J & K to 1



J	K	CLK	Q
0	0	$\uparrow$	Q_0 (no change)
1	0	$\uparrow$	1
0	1	$\uparrow$	0
1	1	$\uparrow$	Q_0 (toggles)

